

DS ORAL

Practical 1: Multi-threaded Client/Server Process Communication using Java RMI	1. What is a distributed system? 2. What is Java RMI? 3. How does RMI differ from socket programming? 4. What is the purpose of the RMI Registry? 5. Explain the architecture of Java RMI. 6. What is a remote object in RMI? 7. What is a remote interface? 8. Why must a remote interface extend Remote? 9. What is the role of RemoteException in RMI? 10. Explain the function of a stub in RMI. 11. What is a skeleton in RMI? 12. What is marshalling and unmarshalling? 13. What is the difference between TCP and UDP? 14. Why is TCP commonly used in RMI? 15. What is multi-threading in Java? 16. Why is multi-threading important in client/server applications? 17. How can mult.	2
## Practical 2: Distributed Application using CORBA	1. What is CORBA? 2. What is the purpose of object brokering? 3. Expand ORB in CORBA. 4. What is an Object Request Broker (ORB)? 5. Explain the architecture of CORBA. 6. What is IDL in CORBA? 7. Why is IDL required? 8. What are the advantages of CORBA? 9. How does CORBA support interoperability? 10. What is a distributed object? 11. Explain static and dynamic invocation in CORBA. 12. What is the role of the client stub in CORBA? 13. What is the role of the server skeleton in CORBA? 14. What are the steps involved in developing a CORBA application? 15. What is naming service in CORBA? 16. How does CORBA differ from Java RMI? 17. What are middleware technologies? 18. What is the role of an.	7
## Practical 3: Sum of N Elements using MPI/OpenMP	1. What is parallel computing? 2. What is distributed computing? 3. What is MPI? 4. Expand MPI. 5. What is OpenMP? 6. Differentiate between MPI and OpenMP. 7. What is process synchronization? 8. What is load balancing? 9. Why is task distribution important? 10. Explain master-slave architecture. 11. What is message passing in MPI? 12. What are communicators in MPI? 13. What is the role of MPI_Init()? 14. What is the role of MPI_Finalize()? 15. Explain point-to-point communication in MPI. 16. What is collective communication? 17. What is speedup in parallel computing? 18. What is scalability? 19. How are intermediate sums calculated in distributed processing? 20. What are race conditions? 2.	13
## Practical 4: Berkeley Algorithm for Clock Synchronization	1. What is clock synchronization? 2. Why is clock synchronization important in distributed systems? 3. Explain the Berkeley algorithm. 4. What is the role of the master node in Berkeley algorithm? 5. How are clock differences calculated? 6. What is clock drift? 7. What are logical clocks? 8. What are physical clocks? 9. What is Cristian's algorithm? 10. Differentiate between Berkeley algorithm and Cristian's algorithm. 11. Why is synchronization necessary in distributed databases? 12. What is network latency? 13. How does network delay affect synchronization? 14. What are the advantages of Berkeley algorithm? 15. What are the limitations of Berkeley algorithm? 16. What happens if.	18
## Practical 5: Token Ring Based Mutual Exclusion Algorithm	1. What is mutual exclusion? 2. Why is mutual exclusion required in distributed systems? 3. Explain the token ring algorithm. 4. What is a critical section? 5. What is a token in token ring algorithm? 6. How is the token passed between processes? 7. What happens when a process receives the token? 8. What are the advantages of token ring algorithm? 9. What are the disadvantages of token ring algorithm? 10. What happens if the token is lost? 11. What is deadlock? 12. What is starvation? 13. How does token ring avoid starvation? 14. Explain fairness in mutual exclusion. 15. What is distributed coordination? 16. Differentiate centralized and distributed mutual exclusion. 17. What are.	23
## Practical 6: Bully and Ring Algorithm for Leader Election	1. What is leader election in distributed systems? 2. Why is leader election important? 3. Explain the Bully algorithm. 4. Why is it called the Bully algorithm? 5. Explain the Ring algorithm. 6. What is a coordinator process? 7. How is process priority decided in Bully algorithm? 8. What happens when the coordinator fails? 9. How does a process detect failure? 10. What are election messages? 11. What is recovery in distributed systems? 12. Compare Bully and Ring algorithms. 13. Which algorithm is more efficient and why? 14. What are the advantages of Bully algorithm? 15. What are the disadvantages of Bully algorithm? 16. What are the advantages of Ring algorithm? 17. What are th.	28
## Practical 7: Web Service and Distributed Application	1. What is a web service? 2. What are the types of web services? 3. Differentiate SOAP and REST. 4. What is REST architecture? 5. What is SOAP protocol? 6. What is WSDL? 7. What is XML? 8. What is JSON? 9. Explain HTTP methods used in REST APIs. 10. What is client-server architecture? 11. What is service-oriented architecture (SOA)? 12. What is the purpose of APIs? 13. What are microservices? 14. What is endpoint in web services? 15. Explain stateless communication. 16. What is request-response communication? 17. What are advantages of RESTful web services? 18. What are disadvantages of SOAP? 19. What is interoperability in web services? 20. How does a client consume a web service? 2.	33



Practical 1: Multi-threaded Client/Server Process Communication using Java RMI

1. What is a distributed system?
2. What is Java RMI?
3. How does RMI differ from socket programming?
4. What is the purpose of the RMI Registry?
5. Explain the architecture of Java RMI.
6. What is a remote object in RMI?
7. What is a remote interface?
8. Why must a remote interface extend `Remote`?
9. What is the role of `RemoteException` in RMI?
10. Explain the function of a stub in RMI.
11. What is a skeleton in RMI?
12. What is marshalling and unmarshalling?
13. What is the difference between TCP and UDP?
14. Why is TCP commonly used in RMI?
15. What is multi-threading in Java?
16. Why is multi-threading important in client/server applications?
17. How can multiple clients communicate with a server simultaneously?
18. What are the advantages of RMI?
19. What are the limitations of RMI?
20. What is JVM and how does it help in distributed computing?
21. Explain the steps involved in creating an RMI application.
22. What is object serialization in Java?
23. What are the security concerns in RMI?
24. What is the difference between local method invocation and remote method invocation?
25. How do you start and stop the RMI Registry?

1. What is a distributed system?

A distributed system is a collection of independent computers that work together as a single system. These computers communicate over a network to share resources, data, and tasks.

2. What is Java RMI?

Java RMI (Remote Method Invocation) is a Java API that allows an object running in one JVM to invoke methods on an object running in another JVM across a network.

3. How does RMI differ from socket programming?

RMI	Socket Programming
High-level API	Low-level communication
Works with objects and methods	Works with raw data streams
Handles serialization automatically	Programmer manages data transfer
Easier for distributed object applications	More flexible but complex

4. What is the purpose of the RMI Registry?

The RMI Registry is a naming service that stores references to remote objects. Clients use it to find and access remote objects by name.

5. Explain the architecture of Java RMI.

Java RMI architecture consists of:

1. **Client**
2. **Stub**
3. **RMI Registry**
4. **Remote Object/Server**
5. **Transport Layer**

The client calls methods on the stub, which communicates with the server object through the network.

6. What is a remote object in RMI?

A remote object is an object whose methods can be invoked from another JVM over a network.

7. What is a remote interface?

A remote interface defines methods that can be called remotely by clients.

Example:

```
Java
public interface MyRemote extends Remote {
    int add(int a, int b) throws RemoteException;
}
```

8. Why must a remote interface extend Remote?

It marks the interface as capable of supporting remote method calls. It informs the JVM that methods can be invoked remotely.

9. What is the role of `RemoteException` in RMI?

`RemoteException` handles communication-related errors such as:

- Network failures
- Server crashes
- Connection issues

All remote methods must declare it.

10. Explain the function of a stub in RMI.

A stub acts as a proxy for the remote object on the client side. It:

- Receives method calls from the client
 - Sends requests to the server
 - Returns results back to the client
-

11. What is a skeleton in RMI?

A skeleton is a server-side component that receives requests from the stub and invokes methods on the actual remote object.

(Note: Modern Java versions generate skeleton functionality dynamically.)

12. What is marshalling and unmarshalling?

- **Marshalling:** Converting objects/data into a byte stream for transmission.
 - **Unmarshalling:** Reconstructing objects/data from the byte stream.
-

13. What is the difference between TCP and UDP?

TCP	UDP
Connection-oriented	Connectionless
Reliable	Faster but unreliable
Error checking available	No guarantee of delivery

TCP	UDP
Ordered data transfer	Unordered transfer

14. Why is TCP commonly used in RMI?

TCP provides:

- Reliable communication
- Ordered data transfer
- Error checking

These features are important for remote method execution.

15. What is multi-threading in Java?

Multithreading is the execution of multiple threads simultaneously within a program to perform multiple tasks concurrently.

16. Why is multi-threading important in client/server applications?

It allows the server to handle multiple client requests at the same time without blocking other clients.

17. How can multiple clients communicate with a server simultaneously?

The server creates separate threads for each client request. In RMI, the JVM automatically manages multiple threads for remote method calls.

18. What are the advantages of RMI?

- Supports distributed computing
- Easy object sharing
- Simplifies network programming
- Built-in security and serialization

- Platform independent (Java-based)
-

19. What are the limitations of RMI?

- Works mainly with Java
 - Slower than local calls
 - Network dependency
 - Firewall/security restrictions
 - More overhead than sockets
-

20. What is JVM and how does it help in distributed computing?

JVM (Java Virtual Machine) executes Java bytecode. It enables platform independence, allowing distributed applications to run on different operating systems.

21. Explain the steps involved in creating an RMI application.

1. Define remote interface
 2. Implement the remote interface
 3. Compile Java files
 4. Start RMI Registry
 5. Register remote object with registry
 6. Create client application
 7. Client looks up remote object and invokes methods
-

22. What is object serialization in Java?

Serialization is the process of converting an object into a byte stream for storage or transmission over a network.

23. What are the security concerns in RMI?

- Unauthorized access
- Code injection
- Data interception

- Remote execution risks

Security can be improved using:

- Security Manager
- SSL
- Authentication mechanisms

24. What is the difference between local method invocation and remote method invocation?

Local Method Invocation	Remote Method Invocation
Same JVM	Different JVMs
Faster	Slower due to network
No serialization needed	Requires serialization
No network dependency	Network required

25. How do you start and stop the RMI Registry?

Start RMI Registry

Bash
rmiregistry

Or on a specific port:

Bash
rmiregistry 1099

Stop RMI Registry

- Press **Ctrl + C** in the terminal where it is running
- Or terminate the process using Task Manager / **kill** command depending on the OS.



Practical 2: Distributed Application using CORBA

1. What is CORBA?
2. What is the purpose of object brokering?
3. Expand ORB in CORBA.
4. What is an Object Request Broker (ORB)?

5. Explain the architecture of CORBA.
6. What is IDL in CORBA?
7. Why is IDL required?
8. What are the advantages of CORBA?
9. How does CORBA support interoperability?
10. What is a distributed object?
11. Explain static and dynamic invocation in CORBA.
12. What is the role of the client stub in CORBA?
13. What is the role of the server skeleton in CORBA?
14. What are the steps involved in developing a CORBA application?
15. What is naming service in CORBA?
16. How does CORBA differ from Java RMI?
17. What are middleware technologies?
18. What is the role of an ORB daemon?
19. Explain parameter passing in CORBA.
20. What are the limitations of CORBA?
21. Why is CORBA considered language-independent?
22. What are the applications of CORBA?
23. What is the difference between distributed objects and local objects?
24. Explain exception handling in CORBA.
25. What is interoperability in distributed systems?

1. What is CORBA?

CORBA (Common Object Request Broker Architecture) is a middleware technology developed by the OMG (Object Management Group) that enables communication between distributed objects written in different programming languages and running on different platforms.

2. What is the purpose of object brokering?

Object brokering helps clients locate and communicate with remote objects transparently, without needing to know their exact location or implementation details.

3. Expand ORB in CORBA.

ORB stands for **Object Request Broker**.

4. What is an Object Request Broker (ORB)?

An ORB is middleware that manages communication between client requests and server objects in a distributed system.

Functions of ORB:

- Locates remote objects
 - Sends requests
 - Transfers data
 - Returns results to clients
-

5. Explain the architecture of CORBA.

CORBA architecture consists of:

1. Client
2. IDL Interface
3. ORB
4. Object Adapter
5. Server Object
6. Naming Service

The client communicates with the server object through the ORB.

6. What is IDL in CORBA?

IDL (Interface Definition Language) is a language-neutral language used to define interfaces for distributed objects.

7. Why is IDL required?

IDL allows applications written in different programming languages to communicate with each other by providing a common interface specification.

8. What are the advantages of CORBA?

- Platform independent
 - Language independent
 - Supports distributed systems
 - Promotes interoperability
 - Scalable and reusable
-

9. How does CORBA support interoperability?

CORBA uses:

- Standardized ORB communication
- IDL interfaces
- Common protocols like IIOP

This allows different systems and languages to communicate seamlessly.

10. What is a distributed object?

A distributed object is an object that exists on a remote machine but can be accessed by clients over a network.

11. Explain static and dynamic invocation in CORBA.

Static Invocation

- Interfaces are known at compile time
- Uses generated stubs and skeletons

Dynamic Invocation

- Interfaces are determined at runtime
 - Uses Dynamic Invocation Interface (DII)
-

12. What is the role of the client stub in CORBA?

The client stub acts as a proxy for the remote object. It:

- Converts method calls into network requests
 - Sends requests to the ORB
 - Receives responses
-

13. What is the role of the server skeleton in CORBA?

The server skeleton:

- Receives client requests from the ORB

- Invokes the actual server methods
- Sends results back to the client

14. What are the steps involved in developing a CORBA application?

1. Define interface using IDL
2. Compile IDL to generate stubs/skeletons
3. Implement server object
4. Create server program
5. Register object with naming service
6. Develop client application
7. Run ORB, server, and client

15. What is naming service in CORBA?

The naming service allows clients to locate remote objects using logical names instead of physical addresses.

16. How does CORBA differ from Java RMI?

CORBA	Java RMI
Language independent	Java specific
Uses IDL	Uses Java interfaces
Supports multiple platforms	Mainly Java-based
More complex	Easier to implement

17. What are middleware technologies?

Middleware technologies are software layers that enable communication and data management between distributed applications.

Examples:

- CORBA
- RMI

- DCOM
 - Web Services
-

18. What is the role of an ORB daemon?

The ORB daemon manages object registration, communication setup, and request routing between clients and servers.

19. Explain parameter passing in CORBA.

CORBA supports:

- **IN parameters** → sent from client to server
 - **OUT parameters** → sent from server to client
 - **INOUT parameters** → modified and returned by server
-

20. What are the limitations of CORBA?

- Complex implementation
 - High overhead
 - Difficult debugging
 - Performance issues
 - Less popular compared to modern technologies
-

21. Why is CORBA considered language-independent?

CORBA uses IDL, which can generate code for multiple programming languages such as Java, C++, Python, and others.

22. What are the applications of CORBA?

CORBA is used in:

- Banking systems
- Telecommunications
- Enterprise applications
- Distributed databases

- Embedded systems

23. What is the difference between distributed objects and local objects?

Distributed Object	Local Object
Exists on remote machine	Exists in same system
Accessed over network	Accessed directly
Slower due to communication overhead	Faster
Requires middleware	No middleware needed

24. Explain exception handling in CORBA.

CORBA handles:

- System exceptions
- User-defined exceptions

Exceptions are used to manage communication failures, invalid requests, and server-side errors.

25. What is interoperability in distributed systems?

Interoperability is the ability of different systems, platforms, or programming languages to communicate and work together effectively.



Practical 3: Sum of N Elements using MPI/OpenMP

1. What is parallel computing?
2. What is distributed computing?
3. What is MPI?
4. Expand MPI.
5. What is OpenMP?
6. Differentiate between MPI and OpenMP.
7. What is process synchronization?
8. What is load balancing?
9. Why is task distribution important?
10. Explain master-slave architecture.
11. What is message passing in MPI?

12. What are communicators in MPI?
13. What is the role of `MPI_Init()`?
14. What is the role of `MPI_Finalize()`?
15. Explain point-to-point communication in MPI.
16. What is collective communication?
17. What is speedup in parallel computing?
18. What is scalability?
19. How are intermediate sums calculated in distributed processing?
20. What are race conditions?
21. What is shared memory architecture?
22. What is distributed memory architecture?
23. What are threads in OpenMP?
24. Explain reduction operation in OpenMP.
25. What are the advantages of parallel processing?

1. What is parallel computing?

Parallel computing is a computing technique in which multiple tasks are executed simultaneously using multiple processors or cores to solve a problem faster.

2. What is distributed computing?

Distributed computing is a model where multiple computers connected through a network work together to complete a task.

3. What is MPI?

MPI is a communication standard used for parallel programming in distributed memory systems.

4. Expand MPI.

MPI stands for **Message Passing Interface**.

5. What is OpenMP?

OpenMP is an API that supports multi-threaded parallel programming in shared memory systems.

6. Differentiate between MPI and OpenMP.

MPI	OpenMP
Uses distributed memory	Uses shared memory
Process-based	Thread-based
Communication through messages	Communication through shared variables
Suitable for clusters	Suitable for multi-core systems

7. What is process synchronization?

Process synchronization is the coordination of multiple processes or threads to ensure correct execution and avoid conflicts during shared resource access.

8. What is load balancing?

Load balancing is the process of distributing work evenly among processors to improve performance and avoid overloading.

9. Why is task distribution important?

Task distribution:

- Improves performance
- Reduces execution time
- Utilizes processors efficiently
- Prevents workload imbalance

10. Explain master-slave architecture.

In master-slave architecture:

- The **master** process assigns tasks
- The **slave/worker** processes execute tasks and return results

11. What is message passing in MPI?

Message passing is a communication method where processes exchange data through send and receive operations.

12. What are communicators in MPI?

Communicators are MPI objects that define a group of processes that can communicate with each other.

Example:

```
C
MPI_COMM_WORLD
```

13. What is the role of MPI_Init()?

MPI_Init() initializes the MPI execution environment before any MPI functions are used.

Example:

```
C
MPI_Init(&argc, &argv);
```

14. What is the role of MPI_Finalize()?

MPI_Finalize() terminates the MPI environment and cleans up resources.

Example:

```
C
MPI_Finalize();
```

15. Explain point-to-point communication in MPI.

Point-to-point communication occurs between two processes using functions like:

- MPI_Send()
 - MPI_Recv()
-

16. What is collective communication?

Collective communication involves all processes in a communicator.

Examples:

- Broadcast
 - Scatter
 - Gather
 - Reduce
-

17. What is speedup in parallel computing?

Speedup measures performance improvement using parallel execution.

Formula:

$$\text{Speedup} = \frac{\text{Sequential Execution Time}}{\text{Parallel Execution Time}}$$

18. What is scalability?

Scalability is the ability of a parallel system to maintain efficiency when the number of processors or workload increases.

19. How are intermediate sums calculated in distributed processing?

Each processor calculates a partial sum for its assigned data. These partial sums are combined to produce the final result.

20. What are race conditions?

Race conditions occur when multiple threads/processes access shared data simultaneously, causing unpredictable results.

21. What is shared memory architecture?

In shared memory architecture, multiple processors access the same memory space.

Example:

- Multi-core systems
-

22. What is distributed memory architecture?

In distributed memory architecture, each processor has its own local memory and communicates using message passing.

Example:

- Computer clusters
-

23. What are threads in OpenMP?

Threads are lightweight execution units created within a process to execute tasks concurrently.

24. Explain reduction operation in OpenMP.

Reduction combines values computed by multiple threads into a single result.

Example:

```
C
#pragma omp parallel for reduction(+:sum)
```

This adds partial sums from all threads into `sum`.

25. What are the advantages of parallel processing?

- Faster computation
- Better CPU utilization
- Solves large complex problems
- Improved performance
- Reduced execution time
- Supports scalability



Practical 4: Berkeley Algorithm for Clock Synchronization

1. What is clock synchronization?
2. Why is clock synchronization important in distributed systems?
3. Explain the Berkeley algorithm.
4. What is the role of the master node in Berkeley algorithm?
5. How are clock differences calculated?
6. What is clock drift?
7. What are logical clocks?
8. What are physical clocks?
9. What is Cristian's algorithm?
10. Differentiate between Berkeley algorithm and Cristian's algorithm.
11. Why is synchronization necessary in distributed databases?
12. What is network latency?
13. How does network delay affect synchronization?
14. What are the advantages of Berkeley algorithm?
15. What are the limitations of Berkeley algorithm?
16. What happens if the master node fails?
17. Explain UTC.
18. What is the difference between internal and external synchronization?
19. What is time skew?

- 20. How frequently should synchronization occur?
- 21. What are real-time distributed systems?
- 22. What is fault tolerance in synchronization algorithms?
- 23. What is NTP?
- 24. Explain distributed clock management.
- 25. How does Berkeley algorithm maintain consistency?

1. What is clock synchronization?

Clock synchronization is the process of coordinating the clocks of multiple computers in a distributed system so they show approximately the same time.

2. Why is clock synchronization important in distributed systems?

It is important for:

- Maintaining event order
 - Coordinating processes
 - Logging events correctly
 - Ensuring consistency in databases and transactions
-

3. Explain the Berkeley algorithm.

The Berkeley algorithm is a clock synchronization algorithm where a master node periodically collects time values from all nodes, calculates the average time difference, and sends adjustments back to synchronize all clocks.

4. What is the role of the master node in Berkeley algorithm?

The master node:

1. Requests current time from all nodes
 2. Computes average clock difference
 3. Calculates adjustments
 4. Sends correction values to each node
-

5. How are clock differences calculated?

The difference is calculated by comparing each node's clock time with the master clock time.

Formula:

$$\text{Clock Difference} = \text{Node Time} - \text{Master Time}$$

6. What is clock drift?

Clock drift is the gradual deviation of a computer's clock from the correct time due to hardware imperfections.

7. What are logical clocks?

Logical clocks are software-based clocks used to maintain the order of events in distributed systems without depending on actual physical time.

Example:

- Lamport Clock
-

8. What are physical clocks?

Physical clocks are real hardware clocks that measure actual calendar or system time.

9. What is Cristian's algorithm?

Cristian's algorithm synchronizes a client clock with a time server by estimating network delay and adjusting the local clock accordingly.

10. Differentiate between Berkeley algorithm and Cristian's algorithm.

Berkeley Algorithm	Cristian's Algorithm
Uses average system time	Uses external time server
No UTC required	Depends on accurate time server
Master adjusts all clocks	Client adjusts its own clock
Suitable for internal synchronization	Suitable for external synchronization

11. Why is synchronization necessary in distributed databases?

Synchronization ensures:

- Correct transaction ordering
 - Data consistency
 - Accurate timestamps
 - Reliable recovery mechanisms
-

12. What is network latency?

Network latency is the time delay in transmitting data between devices over a network.

13. How does network delay affect synchronization?

Network delays can cause incorrect time calculations, leading to inaccurate synchronization among nodes.

14. What are the advantages of Berkeley algorithm?

- Does not require external UTC source
 - Reduces clock differences
 - Suitable for LAN environments
 - Improves internal consistency
-

15. What are the limitations of Berkeley algorithm?

- Single point of failure (master node)
 - Accuracy affected by network delays
 - Not ideal for large-scale systems
 - Requires frequent communication
-

16. What happens if the master node fails?

If the master node fails:

- Synchronization stops
- A new master must be elected
- System may become unsynchronized temporarily

17. Explain UTC.

UTC (Coordinated Universal Time) is the global standard time reference used worldwide for synchronization.

18. What is the difference between internal and external synchronization?

Internal Synchronization	External Synchronization
Synchronizes clocks with each other	Synchronizes clocks with UTC
Focuses on consistency within system	Focuses on real-world accurate time

19. What is time skew?

Time skew is the difference between the clocks of two systems in a distributed environment.

20. How frequently should synchronization occur?

Synchronization frequency depends on:

- Clock drift rate
- Network conditions
- System requirements

Systems with high precision needs synchronize more frequently.

21. What are real-time distributed systems?

Real-time distributed systems are systems where tasks must be completed within strict timing constraints across multiple distributed nodes.

22. What is fault tolerance in synchronization algorithms?

Fault tolerance is the ability of the synchronization system to continue functioning correctly even if some nodes or components fail.

23. What is NTP?

NTP (Network Time Protocol) is a protocol used to synchronize computer clocks over networks using UTC time servers.

24. Explain distributed clock management.

Distributed clock management involves monitoring, synchronizing, and maintaining consistent clocks across multiple distributed systems.

25. How does Berkeley algorithm maintain consistency?

The Berkeley algorithm maintains consistency by:

1. Collecting times from all nodes
2. Computing average time
3. Adjusting all clocks together
4. Minimizing overall clock differences across the system



Practical 5: Token Ring Based Mutual Exclusion Algorithm

1. What is mutual exclusion?
2. Why is mutual exclusion required in distributed systems?
3. Explain the token ring algorithm.
4. What is a critical section?
5. What is a token in token ring algorithm?
6. How is the token passed between processes?
7. What happens when a process receives the token?
8. What are the advantages of token ring algorithm?
9. What are the disadvantages of token ring algorithm?
10. What happens if the token is lost?
11. What is deadlock?
12. What is starvation?
13. How does token ring avoid starvation?
14. Explain fairness in mutual exclusion.

15. What is distributed coordination?
16. Differentiate centralized and distributed mutual exclusion.
17. What are logical topologies?
18. Explain ring topology.
19. What is message complexity in token ring algorithm?
20. What is synchronization overhead?
21. Explain fault tolerance in token ring.
22. What are the applications of mutual exclusion?
23. What is concurrency control?
24. Explain critical resource sharing.
25. What are alternative mutual exclusion algorithms?

1. What is mutual exclusion?

Mutual exclusion is a technique used to ensure that only one process accesses a shared resource or critical section at a time.

2. Why is mutual exclusion required in distributed systems?

It prevents:

- Data inconsistency
 - Race conditions
 - Simultaneous access conflicts
 - Resource corruption
-

3. Explain the token ring algorithm.

The token ring algorithm is a distributed mutual exclusion technique where processes are arranged in a logical ring and a special token circulates among them. Only the process holding the token can enter the critical section.

4. What is a critical section?

A critical section is a part of a program where shared resources or data are accessed.

5. What is a token in token ring algorithm?

A token is a special control message that grants permission to enter the critical section.

6. How is the token passed between processes?

The token is passed sequentially from one process to the next process in the logical ring.

Example:

P1 → P2 → P3 → P4 → P1

7. What happens when a process receives the token?

When a process receives the token:

1. It checks whether it needs the critical section
2. If yes, it enters the critical section
3. After completion, it passes the token to the next process

8. What are the advantages of token ring algorithm?

- Ensures fairness
- Avoids starvation
- Simple coordination
- No centralized controller required

9. What are the disadvantages of token ring algorithm?

- Token loss can stop the system
- Ring failure affects communication
- High waiting time in large systems
- Difficult fault recovery

10. What happens if the token is lost?

If the token is lost:

- No process can enter the critical section
- A token regeneration mechanism must create a new token

11. What is deadlock?

Deadlock is a condition where processes wait indefinitely for resources held by each other.

12. What is starvation?

Starvation occurs when a process waits indefinitely to access the critical section because other processes continuously get access.

13. How does token ring avoid starvation?

Each process eventually receives the token in a fixed order, guaranteeing fair access to the critical section.

14. Explain fairness in mutual exclusion.

Fairness ensures that every process gets an equal opportunity to access the critical section without indefinite waiting.

15. What is distributed coordination?

Distributed coordination is the management of communication and synchronization among multiple distributed processes.

16. Differentiate centralized and distributed mutual exclusion.

Centralized Mutual Exclusion	Distributed Mutual Exclusion
Single coordinator controls access	No central controller
Simpler implementation	More scalable
Single point of failure	Better fault tolerance
Higher bottleneck risk	More communication overhead

17. What are logical topologies?

Logical topologies describe how data flows between nodes in a network regardless of physical connections.

Examples:

- Ring
 - Bus
 - Star
-

18. Explain ring topology.

In ring topology, each node is connected to two neighboring nodes, forming a circular structure for communication.

19. What is message complexity in token ring algorithm?

The message complexity is generally proportional to the number of processes because the token travels through the ring sequentially.

20. What is synchronization overhead?

Synchronization overhead is the extra time and resources required to coordinate processes and maintain consistency.

21. Explain fault tolerance in token ring.

Fault tolerance mechanisms include:

- Token recovery
 - Ring reconfiguration
 - Failure detection
 - Backup token generation
-

22. What are the applications of mutual exclusion?

Applications include:

- Database systems
- File sharing systems

- Distributed transactions
- Printer/resource sharing
- Operating systems

23. What is concurrency control?

Concurrency control manages simultaneous access to shared resources to maintain consistency and correctness.

24. Explain critical resource sharing.

Critical resource sharing ensures that shared resources such as files, memory, or databases are accessed safely without conflicts.

25. What are alternative mutual exclusion algorithms?

Alternative algorithms include:

- Lamport's Algorithm
- Ricart-Agrawala Algorithm
- Centralized Algorithm
- Maekawa's Algorithm
- Raymond's Tree Algorithm



Practical 6: Bully and Ring Algorithm for Leader Election

1. What is leader election in distributed systems?
2. Why is leader election important?
3. Explain the Bully algorithm.
4. Why is it called the Bully algorithm?
5. Explain the Ring algorithm.
6. What is a coordinator process?
7. How is process priority decided in Bully algorithm?
8. What happens when the coordinator fails?
9. How does a process detect failure?
10. What are election messages?
11. What is recovery in distributed systems?
12. Compare Bully and Ring algorithms.

13. Which algorithm is more efficient and why?
14. What are the advantages of Bully algorithm?
15. What are the disadvantages of Bully algorithm?
16. What are the advantages of Ring algorithm?
17. What are the disadvantages of Ring algorithm?
18. What is fault detection?
19. Explain process states in distributed systems.
20. What is message complexity in election algorithms?
21. What are heartbeat messages?
22. Explain distributed consensus.
23. What happens if multiple elections occur simultaneously?
24. What is scalability in election algorithms?
25. What are real-world applications of leader election?

1. What is leader election in distributed systems?

Leader election is the process of selecting one process as the coordinator or leader among multiple distributed processes.

2. Why is leader election important?

Leader election is important for:

- Coordinating tasks
 - Managing shared resources
 - Synchronization
 - Decision making
 - Fault management
-

3. Explain the Bully algorithm.

In the Bully algorithm:

1. A process detects coordinator failure
 2. It sends election messages to higher-priority processes
 3. The highest active process becomes the new coordinator
 4. The new coordinator announces itself to all processes
-

4. Why is it called the Bully algorithm?

It is called the Bully algorithm because the process with the highest priority “bullies” lower-priority processes and becomes the coordinator.

5. Explain the Ring algorithm.

In the Ring algorithm:

1. Processes are arranged logically in a ring
 2. An election message circulates through the ring
 3. Each process adds its ID to the message
 4. The process with the highest ID becomes the coordinator
-

6. What is a coordinator process?

A coordinator process is the elected leader responsible for controlling or coordinating distributed operations.

7. How is process priority decided in Bully algorithm?

Priority is usually determined using process IDs. Higher process ID means higher priority.

8. What happens when the coordinator fails?

When the coordinator fails:

- Failure is detected by other processes
 - An election process begins
 - A new coordinator is selected
-

9. How does a process detect failure?

A process detects failure using:

- Timeout mechanisms
 - Missing heartbeat messages
 - Communication errors
-

10. What are election messages?

Election messages are communication messages exchanged between processes during the leader election process.

11. What is recovery in distributed systems?

Recovery is the process of restoring normal system operation after failures.

12. Compare Bully and Ring algorithms.

Bully Algorithm	Ring Algorithm
Uses process priorities	Uses ring structure
Faster in small systems	Simpler communication
High message overhead	Lower message overhead
Requires knowledge of all processes	Requires logical ring

13. Which algorithm is more efficient and why?

The Ring algorithm is generally more message-efficient because messages travel sequentially in the ring, while the Bully algorithm may generate many election messages.

14. What are the advantages of Bully algorithm?

- Fast coordinator selection
 - Simple implementation
 - High-priority process always becomes leader
-

15. What are the disadvantages of Bully algorithm?

- High message complexity
 - More network traffic
 - Poor scalability
 - Requires knowledge of all processes
-

16. What are the advantages of Ring algorithm?

- Lower message overhead
 - Simple structure
 - Better scalability than Bully algorithm
-

17. What are the disadvantages of Ring algorithm?

- Election delay due to ring traversal
 - Ring failure can disrupt communication
 - Complex ring maintenance
-

18. What is fault detection?

Fault detection is the identification of failed processes or communication links in a distributed system.

19. Explain process states in distributed systems.

Common process states include:

- Active
 - Idle
 - Failed
 - Recovering
 - Coordinator state
-

20. What is message complexity in election algorithms?

Message complexity refers to the number of messages exchanged during the election process.

21. What are heartbeat messages?

Heartbeat messages are periodic signals sent by processes to indicate they are active and functioning.

22. Explain distributed consensus.

Distributed consensus is the process by which distributed processes agree on a common value or decision.

23. What happens if multiple elections occur simultaneously?

Simultaneous elections may:

- Increase message traffic
- Cause temporary conflicts

Election algorithms eventually resolve conflicts and elect one coordinator.

24. What is scalability in election algorithms?

Scalability is the ability of an election algorithm to perform efficiently as the number of processes increases.

25. What are real-world applications of leader election?

Applications include:

- Distributed databases
- Cloud computing
- Cluster management
- Distributed file systems
- Network routing systems



Practical 7: Web Service and Distributed Application

1. What is a web service?
2. What are the types of web services?
3. Differentiate SOAP and REST.
4. What is REST architecture?
5. What is SOAP protocol?
6. What is WSDL?
7. What is XML?
8. What is JSON?
9. Explain HTTP methods used in REST APIs.
10. What is client-server architecture?

11. What is service-oriented architecture (SOA)?
12. What is the purpose of APIs?
13. What are microservices?
14. What is endpoint in web services?
15. Explain stateless communication.
16. What is request-response communication?
17. What are advantages of RESTful web services?
18. What are disadvantages of SOAP?
19. What is interoperability in web services?
20. How does a client consume a web service?
21. What is API testing?
22. What are HTTP status codes?
23. What is serialization in web services?
24. What is middleware in distributed applications?
25. Explain security mechanisms in web services.

1. What is a web service?

A web service is a software system that enables communication and data exchange between applications over a network using standard web protocols.

2. What are the types of web services?

Main types of web services:

1. SOAP Web Services
2. RESTful Web Services

3. Differentiate SOAP and REST.

SOAP	REST
Protocol-based	Architectural style
Uses XML only	Supports JSON, XML, etc.
More secure and standardized	Lightweight and faster
Higher overhead	Easier to implement

4. What is REST architecture?

REST (Representational State Transfer) is an architectural style for developing web services using HTTP methods and stateless communication.

5. What is SOAP protocol?

SOAP (Simple Object Access Protocol) is a protocol for exchanging structured information using XML over networks.

6. What is WSDL?

WSDL (Web Services Description Language) is an XML-based document that describes the functionality and interface of a SOAP web service.

7. What is XML?

XML (Extensible Markup Language) is a markup language used to store and transport structured data.

Example:

```
XML
<student>
  <name>Aniket</name>
</student>
```

8. What is JSON?

JSON (JavaScript Object Notation) is a lightweight data format used for data exchange.

Example:

```
JSON
{
  "name": "Aniket"
}
```

9. Explain HTTP methods used in REST APIs.

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Remove data

10. What is client-server architecture?

Client-server architecture is a model where clients request services and servers provide resources or responses.

11. What is service-oriented architecture (SOA)?

SOA is an architectural approach where applications are built using reusable and loosely coupled services.

12. What is the purpose of APIs?

APIs (Application Programming Interfaces) allow different software applications to communicate and exchange data.

13. What are microservices?

Microservices are small independent services that work together to form a larger application.

14. What is endpoint in web services?

An endpoint is a specific URL where a web service can be accessed.

Example:

`https://api.example.com/users`

15. Explain stateless communication.

In stateless communication, each client request is independent and the server does not store previous request information.

16. What is request-response communication?

It is a communication model where:

1. Client sends a request
 2. Server processes it
 3. Server returns a response
-

17. What are advantages of RESTful web services?

- Lightweight
 - Faster communication
 - Easy integration
 - Supports multiple data formats
 - Scalable
-

18. What are disadvantages of SOAP?

- Complex implementation
 - High bandwidth usage
 - Slower performance
 - XML processing overhead
-

19. What is interoperability in web services?

Interoperability is the ability of different systems and applications to communicate regardless of platform or programming language.

20. How does a client consume a web service?

A client consumes a web service by:

1. Sending HTTP requests
 2. Receiving responses
 3. Processing returned data
-

21. What is API testing?

API testing verifies whether APIs function correctly, securely, and efficiently.

22. What are HTTP status codes?

HTTP status codes indicate the result of an HTTP request.

Examples:

Code	Meaning
200	Success
404	Not Found
500	Server Error

23. What is serialization in web services?

Serialization is the process of converting objects or data into a transferable format such as JSON or XML.

24. What is middleware in distributed applications?

Middleware is software that enables communication and coordination between distributed applications.

Examples:

- CORBA
 - RMI
 - Web Servers
-

25. Explain security mechanisms in web services.

Security mechanisms include:

- Authentication
- Authorization
- Encryption
- SSL/TLS
- API keys
- OAuth tokens